

Automated Data Migration Process Using CSV

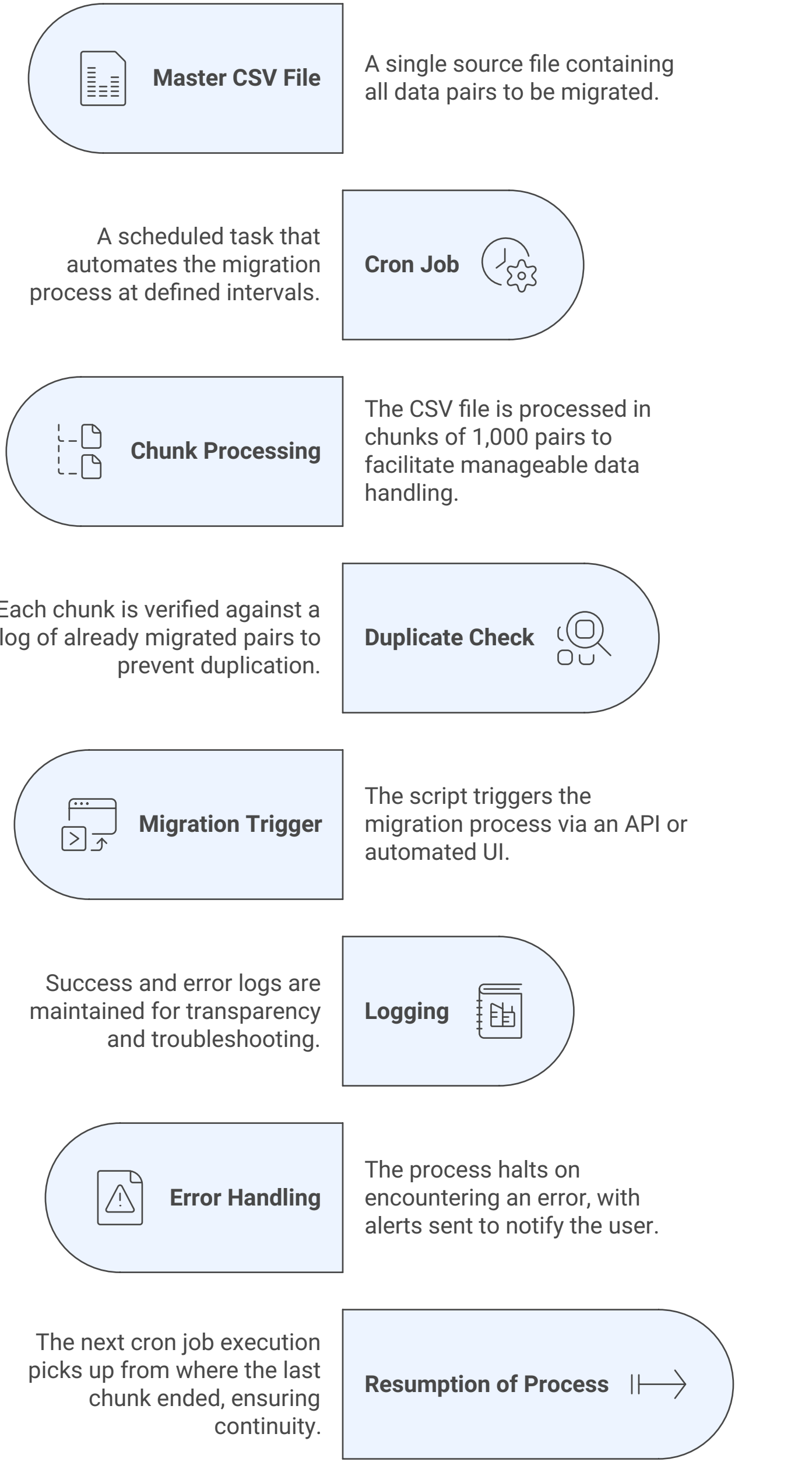
This document outlines a robust logic framework for automating the migration of data stored in a master CSV file. The process is designed to efficiently handle large datasets, ensuring that no duplicates are created and that errors are managed effectively. By utilizing a cron job to manage the migration in manageable chunks, this approach enhances reliability and maintainability.

Overview of the Migration Process

The migration process involves several key components:

- Master CSV File:** A single source file containing all data pairs to be migrated.
- Cron Job:** A scheduled task that automates the migration process at defined intervals.
- Chunk Processing:** The CSV file is processed in chunks of 1,000 pairs to facilitate manageable data handling.
- Duplicate Check:** Each chunk is verified against a log of already migrated pairs to prevent duplication.
- Migration Trigger:** The script triggers the migration process via an API or automated UI.
- Logging:** Success and error logs are maintained for transparency and troubleshooting.
- Error Handling:** The process halts on encountering an error, with alerts sent to notify the user.
- Resumption of Process:** The next cron job execution picks up from where the last chunk ended, ensuring continuity.

Migration Process Components



Detailed Process Flow

1. Master CSV File

The master CSV file serves as the central repository for all data pairs. It should be structured properly, with clear headers and consistent data formats to facilitate smooth processing.

2. Cron Job Setup

A cron job is configured to run at specified intervals [e.g., every hour]. This job will execute a script that handles the migration process. The cron job ensures that the migration is automated and does not require manual intervention.

3. Chunk Processing

The script reads the master CSV file in chunks of 1,000 pairs. This chunk size is chosen to balance performance and manageability. Processing in chunks helps to avoid memory overload and allows for easier error handling.

4. Duplicate Check

Before migrating each chunk, the script checks against a log of already migrated pairs. This log is updated after each successful migration. The duplicate check is crucial to ensure that no data is migrated more than once, maintaining data integrity.

5. Migration Trigger

For each chunk, the script triggers the migration process. This can be done through:

- API Calls:** If the destination system supports API integration, the script can send the data directly.
- Automated UI:** If an API is not available, the script can simulate user actions to input the data into the destination system.

6. Logging

The script maintains two types of logs:

- Success Log:** Records each successfully migrated pair along with a timestamp.
- Error Log:** Captures any errors encountered during the migration process, including error messages and the specific chunk that caused the issue.

7. Error Handling

If an error occurs during the migration of a chunk, the process is halted immediately. The script sends an alert [via email or another notification system] to inform the user of the issue. This immediate feedback allows for quick troubleshooting and resolution.

8. Resumption of Process

On the next execution of the cron job, the script checks the success log to determine where the last successful migration ended. It then resumes processing from that point, ensuring that all 50,000 pairs are eventually migrated without duplication or data loss.

Conclusion

This automated data migration process provides a structured and efficient approach to handling large datasets stored in a master CSV file. By implementing chunk processing, duplicate checks, and robust error handling, the system ensures that data integrity is maintained throughout the migration. The use of a cron job for automation further enhances the reliability of the process, allowing for seamless data migration with minimal manual intervention.